Computability – Topic 5

Yonghao Lee

February 2026

Contents

1 Algorithms and Turing Machines **2**
1.1 Hilbert's Tenth Problem . . . 2
1.2 Terminology for Describing Turing Machines . . . 2
1.3 Example: Graph Connectivity . . . 3

2 Decidability **3**
2.1 Acceptance Problem for DFAs . . . 3
2.2 Acceptance Problem for NFAs . . . 4
2.3 Acceptance Problem for Regular Expressions . . . 4
2.4 Emptiness Testing for DFAs . . . 5
2.5 Equivalence Testing for DFAs . . . 5
2.6 Decidable Problems Concerning Context-Free Grammars . . . 6
2.7 Summary of Decidability Results . . . 7

3 The Hierarchy of Language Classes **7**

4 The Class RE, Enumerators, and Dovetailing **8**
4.1 What is an Enumerator? . . . 8
4.2 Enumerating Σ^* in Minlex Order . . . 9
4.3 Zig-Zag Enumerators and Decidability . . . 10
4.4 Dovetailing . . . 11
4.4.1 Application 1: RE is Closed Under Kleene Star . . . 12
4.4.2 Application 2: Recognizing $\mathrm{NE}_{\mathrm{TM}}$ . . . 12

1 Algorithms and Turing Machines

An **algorithm** is a collection of simple instructions for carrying out some task. The notion of an algorithm itself was not defined precisely until the twentieth century.

1.1 Hilbert's Tenth Problem

We look at the following problem:

$$D = \{p \mid p \text{ is a polynomial with an integral root}\}$$

This language is **not decidable**; yet, it is **RE** (recursively enumerable). For simplicity, let us first look at a restricted version of the problem:

$$D_1 = \{p \mid p \text{ is a polynomial over } x \text{ with an integral root}\}$$

TM M_1 recognizing D_1

M_1 = "On input $\langle p \rangle$, where p is a polynomial over the variable x:
1. Evaluate p with x set successively to the values $0, 1, -1, 2, -2, 3, -3, \ldots$
2. If at any point the polynomial evaluates to 0, **accept**."

If p has an integral root, M_1 will eventually find it and accept. If p does not have an integral root, then M_1 will run forever — this is precisely why M_1 is a *recognizer* but not a *decider*.

Remark 1.1: Converting M_1 to a Decider

For the single-variable case, we *can* convert M_1 into a decider, since we can calculate bounds within which the roots of a single-variable polynomial must lie and restrict the search to those bounds.

For the multivariable case, we can present a similar TM that *recognizes* D: the machine M goes through all possible assignments of its variables to integer values (using dovetailing to ensure every assignment is eventually tried). However, Matiyasevich's theorem (1970) showed that no decider for D exists — the language D is undecidable.

1.2 Terminology for Describing Turing Machines

We continue to speak of Turing machines, but our real focus from now on is on **algorithms**. The TM merely serves as a precise model for the definition of algorithms. We only need to be comfortable enough with TMs to believe that they capture all algorithms (the Church–Turing thesis).

There are three levels of description for TMs:

1. **Formal description:** Spells out in full the TM's states, transition function, and so on. This is the most detailed and precise level.
2. **Implementation description:** Uses English prose to describe the way the Turing machine moves its head and the way it stores data on its tape. No explicit state diagram is given.
3. **High-level description:** Uses English prose to describe an algorithm, ignoring the implementation details entirely. This is the level we use most often.

Remark 1.2: Encoding Objects as Strings

The input to a TM is always a *string*. If we want to provide an object other than a string as input, we must first represent that object as a string. Strings can easily represent polynomials, graphs, grammars, automata, and so on.
We use **angle brackets** $\langle \cdot \rangle$ to denote the encoding of an object as a string. For example, $\langle G \rangle$ denotes the encoding of a graph G, and $\langle M, w \rangle$ denotes the encoding of a TM M together with an input string w.

1.3 Example: Graph Connectivity

Example 1.1: Deciding Graph Connectivity

Let A be the language consisting of all strings representing undirected graphs that are connected. Recall that a graph is *connected* if every node can be reached from every other node by travelling along the edges of the graph.

$$A = \{\langle G \rangle \mid G \text{ is a connected undirected graph}\}$$

We present a TM M that **decides** A:

TM M deciding A

M = "On input $\langle G \rangle$, the encoding of a graph G:

1. Select the first node of G and mark it.
2. Repeat the following stage until no new nodes are marked:
   a. For each node in G, mark it if it is attached by an edge to a node that is already marked.
3. Scan all nodes of G to determine whether they are all marked. If they are, **accept**; otherwise, **reject**."

Remark 1.3: Why M is a Decider

This machine always halts: in each iteration of step 2, either at least one new node gets marked (and there are finitely many nodes), or no new node is marked and the loop terminates. After the loop, step 3 makes a definitive accept/reject decision. Therefore M is a *decider* for A, not merely a recognizer.

2 Decidability

We now demonstrate that certain problems can be solved algorithmically, while others cannot. We study the unsolvability of problems because knowing a problem is unsolvable tells us that it must be simplified or altered before we can find an algorithmic solution.

We begin with computational problems concerning **finite automata**. We give algorithms for testing whether a finite automaton accepts a string, whether its language is empty, and whether two finite automata are equivalent.

2.1 Acceptance Problem for DFAs

Definition 2.1: A_{DFA}

$$A_{\text{DFA}} = \{\langle B, w \rangle \mid B \text{ is a DFA that accepts input string } w\}$$

The problem of testing whether a DFA B accepts an input w is the same as the problem of testing whether $\langle B, w \rangle$ is a member of the language A_{DFA}.

Theorem 2.1: A_{DFA} is Decidable

A_{DFA} is a decidable language.

Proof. We present a TM M that decides A_{DFA}.

TM M deciding A_{DFA}

M = "On input $\langle B, w\rangle$, where B is a DFA and w is a string:

1. Verify that $\langle B, w\rangle$ properly encodes a DFA B and a string w. If not, **reject**.
2. Simulate B on input w:
   a. Initialize the current state to q_0 and the current input position to the leftmost symbol of w.
   b. For each symbol in w, update the current state according to B's transition function δ.
3. After processing the last symbol of w: if B is in an accepting state, **accept**; otherwise, **reject**."

Correctness. If B accepts w, then the simulation ends in an accepting state and M accepts. If B does not accept w, then the simulation ends in a non-accepting state and M rejects.

Halting. The simulation processes exactly $|w|$ symbols and then halts, so M is a decider. □

2.2 Acceptance Problem for NFAs

Definition 2.2: A_{NFA}

$$A_{\text{NFA}} = \{\langle B, w\rangle \mid B \text{ is an NFA that accepts input string } w\}$$

Theorem 2.2: A_{NFA} is Decidable

A_{NFA} is a decidable language.

Proof. We present a TM N that decides A_{NFA}.

TM N deciding A_{NFA}

N = "On input $\langle B, w\rangle$, where B is an NFA and w is a string:

1. Convert NFA B into an equivalent DFA C using the subset construction.
2. Run TM M (the decider for A_{DFA}) on input $\langle C, w\rangle$.
3. If M accepts, **accept**; if M rejects, **reject**."

Correctness. The subset construction produces a DFA C with $L(C) = L(B)$. Therefore $w \in L(B)$ iff C accepts w, which is what M decides.

Halting. The subset construction is a finite procedure, and M is a decider. □

2.3 Acceptance Problem for Regular Expressions

Definition 2.3: A_{REX}

$$A_{\text{REX}} = \{\langle R, w\rangle \mid R \text{ is a regular expression that generates string } w\}$$

Theorem 2.3: A_{REX} is Decidable

A_{REX} is a decidable language.

Proof. We present a TM P that decides A_{REX}.

TM P deciding A_{REX}

P = "On input $\langle R, w\rangle$, where R is a regular expression and w is a string:

1. Convert R into an equivalent NFA A using the procedure for converting regular expressions to NFAs.
2. Run TM N (the decider for A_{NFA}) on input $\langle A, w\rangle$.
3. If N accepts, **accept**; if N rejects, **reject**."

Correctness. The conversion in step 1 produces an NFA A such that $L(A) = L(R)$. Therefore $w \in L(R)$ if and only if A accepts w, which is precisely what N decides in step 2.

Halting. Step 1 is a finite conversion procedure, and N is a decider (always halts), so P halts on every input. □

Remark 2.1: Reduction Chain for Acceptance Problems

Notice the chain of reductions:

$$A_{\text{REX}} \longrightarrow A_{\text{NFA}} \longrightarrow A_{\text{DFA}}$$

Each problem is reduced to a previously solved one. We convert the input to a simpler computational model, then use the existing decider for that model. This is a recurring proof pattern.

2.4 Emptiness Testing for DFAs

Definition 2.4: E_{DFA}

$$E_{\text{DFA}} = \{\langle A\rangle \mid A \text{ is a DFA and } L(A) = \emptyset\}$$

Theorem 2.4: E_{DFA} is Decidable

E_{DFA} is a decidable language.

Proof. A DFA accepts some string if and only if an accept state is reachable from the start state by following the transitions of the DFA. We test this condition using a marking algorithm.

TM T deciding E_{DFA}

T = "On input $\langle A\rangle$, where A is a DFA:

1. Mark the start state of A.
2. Repeat until no new states get marked:
   a. Mark any state that has a transition coming into it from a state that is already marked.
3. If no accept state is marked, **accept**; otherwise, **reject**."

Correctness. After the loop terminates, a state is marked if and only if it is reachable from q_0. If no accept state is reachable, then no string drives A to an accept state, so $L(A) = \emptyset$ and T correctly accepts. If some accept state is reachable, then $L(A) \neq \emptyset$ and T correctly rejects.

Halting. Each iteration of the loop marks at least one new state. Since A has finitely many states, the loop terminates after at most $|Q|$ iterations. □

2.5 Equivalence Testing for DFAs

Definition 2.5: EQ_{DFA}

$$\text{EQ}_{\text{DFA}} = \{\langle A, B\rangle \mid A, B \text{ are DFAs and } L(A) = L(B)\}$$

Theorem 2.5: $\mathrm{EQ}_{\mathrm{DFA}}$ is Decidable

$\mathrm{EQ}_{\mathrm{DFA}}$ is a decidable language.

Proof. The key idea is to use the **symmetric difference**. We construct a new DFA C such that:

$$L(C) = \big(L(A) \cap \overline{L(B)}\big) \cup \big(\overline{L(A)} \cap L(B)\big)$$

This is the set of strings accepted by exactly one of A or B, but not both. Observe that $L(A) = L(B)$ if and only if $L(C) = \emptyset$.

TM F deciding $\mathrm{EQ}_{\mathrm{DFA}}$

F = "On input $\langle A, B\rangle$, where A and B are DFAs:

1. Construct DFA C recognizing the symmetric difference $L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$.
2. Run TM T (the decider for E_{DFA}) on input $\langle C\rangle$.
3. If T accepts, **accept**; if T rejects, **reject**."

Correctness. $L(A) = L(B)$ if and only if no string is accepted by one but not the other, i.e., $L(C) = \emptyset$. This is exactly what T decides.

Construction of C. We need to show that we can explicitly build a DFA for the symmetric difference given the DFAs A and B:

- To complement a DFA, swap its accept and non-accept states. This gives DFAs for $\overline{L(A)}$ and $\overline{L(B)}$.
- To intersect or union two DFAs, use the product construction.

Composing these constructions yields the DFA C. All steps are effective procedures that produce a concrete DFA.

Halting. Step 1 is a finite construction, and T is a decider, so F halts on every input. □

2.6 Decidable Problems Concerning Context-Free Grammars

Definition 2.6: A_{CFG}

$$A_{\mathrm{CFG}} = \{\langle G, w\rangle \mid G \text{ is a CFG that generates string } w\}$$

Theorem 2.6: A_{CFG} is Decidable

A_{CFG} is a decidable language.

Idea. A naive approach is to use G to exhaustively go through all possible derivations and check whether any of them produces w. But this does not work directly: if G does not generate w, we may have to try infinitely many derivations and the algorithm would never halt.

To make this into a decider, we need to bound the number of derivations to check. The key insight is to first convert G into **Chomsky Normal Form (CNF)**. In CNF, every derivation of a string w of length $n \geq 1$ has exactly $2n - 1$ steps. This means we only need to check finitely many derivations.

Proof. We present a TM S that decides A_{CFG}.

TM S deciding A_{CFG}

S = "On input $\langle G, w\rangle$, where G is a CFG and w is a string:

1. Convert G to an equivalent grammar G' in Chomsky Normal Form.
2. If $w = \varepsilon$: check whether the start variable has the rule $S \to \varepsilon$ in G'. If so, **accept**; otherwise, **reject**.
3. If $w \neq \varepsilon$: list all derivations in G' with exactly $2n - 1$ steps, where $n = |w|$.
4. If any of these derivations generate w, **accept**; otherwise, **reject**."

Correctness. After conversion to CNF, $L(G') = L(G)$. For the case $w \neq \varepsilon$: in CNF, every derivation of a string of length n uses exactly $2n-1$ steps, so enumerating all such derivations is guaranteed to find a derivation of w if one exists. For the case $w = \varepsilon$: in CNF, the only way to generate ε is via $S \to \varepsilon$, so step 2 handles this correctly.

Halting. The conversion to CNF is a finite procedure. There are finitely many derivations of length $2n - 1$ (since at each step we choose from finitely many rules). Therefore S halts on every input. □

Remark 2.2: Why the Special Case for $w = \varepsilon$?

In Chomsky Normal Form, every rule is of the form $A \to BC$ or $A \to a$ (where A, B, C are variables and a is a terminal). The only exception is $S \to \varepsilon$, which is allowed only if S is the start variable and S does not appear on the right-hand side of any rule. Since $S \to \varepsilon$ is a derivation of length 1 (not $2(0) - 1 = -1$), the formula $2n - 1$ does not apply when $n = 0$, and we must handle this case separately.

2.7 Summary of Decidability Results

Language	Decidable?	Proof Technique
A_{DFA}	✓	Direct simulation
A_{NFA}	✓	Convert to DFA, then use A_{DFA} decider
A_{REX}	✓	Convert to NFA, then use A_{NFA} decider
E_{DFA}	✓	Marking / reachability algorithm
EQ_{DFA}	✓	Symmetric difference + E_{DFA} decider
A_{CFG}	✓	Convert to CNF, bound derivation length

3 The Hierarchy of Language Classes

The following diagram shows the containment relationships among the major classes of languages. Each class is strictly contained in the next.

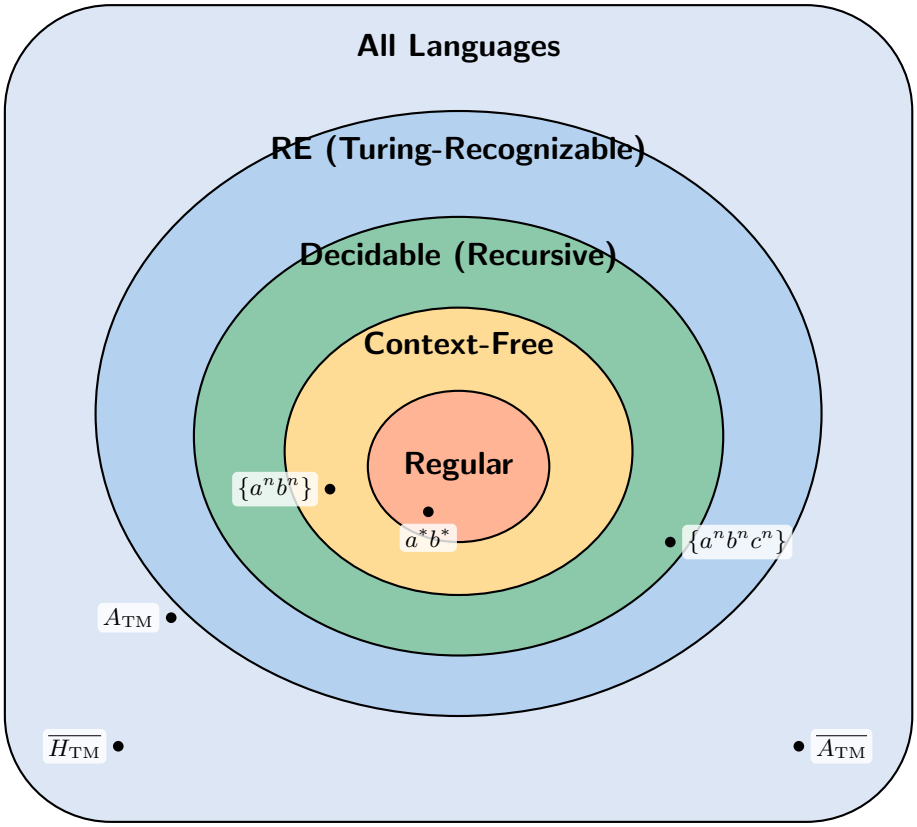

Figure 1: The relationship among classes of languages.

4 The Class RE, Enumerators, and Dovetailing

4.1 What is an Enumerator?

Definition 4.1: Enumerator

An **enumerator** for a language L is a Turing machine E with a special "print" operation. The machine E takes *no input*. It simply runs and, from time to time, prints strings to an output tape. The language **enumerated** by E is the set of all strings that E eventually prints:

$$L(E) = \{w \in \Sigma^* \mid E \text{ prints } w \text{ at some point during its execution}\}$$

The strings may be printed in any order, and repetitions are allowed.

Theorem 4.1: Enumerator Characterization of RE

A language L is recursively enumerable (RE) if and only if some enumerator enumerates it.

Proof. ($\Rightarrow$) **If L is RE, then L has an enumerator.**

Let M be a TM that recognizes L. We build an enumerator E as follows:

Enumerator E for L

E = "Ignore the input (enumerators don't use input).

1. For $i = 1, 2, 3, \ldots$:
   a. Let $s_1, s_2, \ldots, s_i$ be the first i strings in Σ^* (in minlex order).
   b. Run M on each of $s_1, s_2, \ldots, s_i$ for i steps each.
   c. For any s_j that M accepted within those i steps, print s_j."

Why does this work? If $w \in L$, then M accepts w in some finite number of steps t. Let $w = s_k$ (the k-th string in minlex order). Then when $i \geq \max(k, t)$, step (a) includes w in the list and step (b) runs M long enough to accept. So E eventually prints w.

Why can't we just run M on s_1 to completion, then s_2, then s_3, etc.? Because M might *loop forever* on s_1 and we would never get to s_2. The trick of running all strings for increasingly many steps is called **dovetailing** (see Section 4.4).

($\Leftarrow$) **If L has an enumerator, then L is RE.**

Let E be an enumerator for L. We build a TM M that recognizes L:

TM M recognizing L from enumerator E

M = "On input w:
1. Simulate E.
2. Every time E prints a string s, compare s with w.
3. If $s = w$, **accept**."

If $w \in L$, then E eventually prints w, so M will find it and accept. If $w \notin L$, then E never prints w, so M runs forever — exactly RE behavior. □

4.2 Enumerating Σ^* in Minlex Order

As a warm-up, we show that for any finite alphabet $\Sigma = \{a_1, a_2, \ldots, a_k\}$ with $a_1 < a_2 < \cdots < a_k$, the set Σ^* can be enumerated in **minlex order**.

Definition 4.2: Minlex Order

For $x, y \in \Sigma^*$, we say $x <_{\text{minlex}} y$ if:
- $|x| < |y|$ (shorter strings come first), or
- $|x| = |y|$ and x is lexicographically smaller than y.

For example, over $\{a, b\}$: $\varepsilon <_{\text{minlex}} a <_{\text{minlex}} b <_{\text{minlex}} aa <_{\text{minlex}} ab <_{\text{minlex}} ba <_{\text{minlex}} bb <_{\text{minlex}} aaa <_{\text{minlex}} \cdots$

Idea. Think of strings over $\Sigma = \{a_1, \ldots, a_k\}$ as numbers in base k. Going to the "next" string in minlex order is like incrementing a number by 1, with carrying. When all digits overflow (e.g., $bb \to aaa$), the string grows by one character.

TM T computing the next string in minlex order

T = "On input $w \in \Sigma^*$ (the current string):
1. If the tape is empty, write a_1 and halt.
2. Mark the first cell (to remember where the string starts).
3. Move to the *rightmost* character of w (the least significant "digit").
4. **Increment with carry:**
   a. If the current symbol is a_i with $i < k$: replace it with a_{i+1} and go to step 5. *(No carry needed.)*
   b. If the current symbol is a_k: replace it with a_1 and move left. *(Carry!)* Repeat step 4.
   c. If the carry propagates all the way to the marked first cell and it is also a_k: replace it with a_1, move right to the first blank cell, write a_1. *(The string grew by one character.)* Go to step 5.
5. Unmark the first cell and halt. The tape now contains the next string in minlex order."

Example 4.1: Incrementing over $\Sigma = \{a, b, c\}$

Current string	$\longrightarrow$	Next string	
ε	$\longrightarrow$	a	
a	$\longrightarrow$	b	
b	$\longrightarrow$	c	
c	$\longrightarrow$	aa	(carry overflows, string grows)
ab	$\longrightarrow$	ac	
ac	$\longrightarrow$	ba	(carry propagates one position)
cc	$\longrightarrow$	aaa	(full overflow, string grows)

Remark 4.1: Why This Matters

Being able to enumerate Σ^* is a fundamental building block. Many proofs in computability theory need to "loop over all strings" — for instance, the dovetailing technique and the Zig-Zag enumerator both rely on having a systematic way to list every string.

4.3 Zig-Zag Enumerators and Decidability

This is a result that has appeared on previous exams. It connects a special kind of enumerator to decidability.

Definition 4.3: Zig-Zag Enumerator

A **Zig-Zag Enumerator** for a language $L \subseteq \Sigma^*$ is an enumerator that prints *every* string in Σ^* (not just those in L), with the following constraint:

- Strings in L are printed in **even** positions (2nd, 4th, 6th, ...).
- Strings *not* in L are printed in **odd** positions (1st, 3rd, 5th, ...).

Intuition. A Zig-Zag Enumerator "interleaves" membership information into the enumeration order. By looking at *where* a string appears (even or odd position), you can determine whether it belongs to L. This is a very strong property.

Theorem 4.2: Zig-Zag Enumerator $\iff$ Decidable + Non-trivial

A language L has a Zig-Zag Enumerator if and only if:

(i) $L \in R$ (i.e., L is decidable), and

(ii) L is non-trivial (i.e., $L \neq \emptyset$ and $L \neq \Sigma^*$).

Proof. **($\Rightarrow$) Zig-Zag Enumerator implies $L \in R$ and non-trivial.**

Let E be a Zig-Zag Enumerator for L. We build a decider D for L:

Decider D for L using Zig-Zag Enumerator E

$D =$ "On input w:

1. Simulate E and maintain a counter $n = 1, 2, 3, \ldots$ tracking which position is being printed.
2. Each time E prints a string s:
   a. If $s = w$ and n is even: **accept** (because w appeared in an even position, so $w \in L$).
   b. If $s = w$ and n is odd: **reject** (because w appeared in an odd position, so $w \notin L$).
3. Increment n and continue."

Why this halts: E prints every string in Σ^*, so it will eventually print w. At that point D halts with the correct answer. Therefore $L \in R$.

Non-triviality: Since E prints strings in both even and odd positions, there must exist at least one string in L (occupying an even slot) and at least one string not in L (occupying an odd slot). So $L \neq \emptyset$ and $L \neq \Sigma^*$.

($\Leftarrow$) $L \in R$ and non-trivial implies L has a Zig-Zag Enumerator.

Let D be a decider for L. Since L is non-trivial, there exist $w_{\text{in}} \in L$ and $w_{\text{out}} \notin L$. We construct a Zig-Zag Enumerator E.

Initialization: E enumerates Σ^* in minlex order, running D on each string, until it finds one string $w_{\text{out}} \notin L$ and one string $w_{\text{in}} \in L$. (This succeeds because L is non-trivial and D always halts.)

Main loop: E maintains a parity bit b (initialized to 1, meaning "the next slot is odd"). Then, for each string w in Σ^* (in minlex order), E does the following:

Zig-Zag Enumerator E

For each w in Σ^* in minlex order:

1. Run D on w.
2. **If D accepts** ($w \in L$), we need w in an even slot:
   - If $b = 1$ (next slot is odd): print w_{out} first (fills the odd slot with a non-L word), then print w (now in the even slot). Set $b \leftarrow 1$.
   - If $b = 0$ (next slot is even): print w directly. Set $b \leftarrow 1$.
3. **If D rejects** ($w \notin L$), we need w in an odd slot:
   - If $b = 0$ (next slot is even): print w_{in} first (fills the even slot with an L word), then print w (now in the odd slot). Set $b \leftarrow 1$.
   - If $b = 1$ (next slot is odd): print w directly. Set $b \leftarrow 0$.

Why this works: Every string $w \in \Sigma^*$ gets printed exactly once (in the main loop). The "padding" words w_{in} and w_{out} ensure that the parity is always correct: words in L land on even positions, words not in L land on odd positions. The padding words may be printed multiple times (which is allowed for enumerators). $\square$

4.4 Dovetailing

Definition 4.4: Dovetailing (Informal)

Dovetailing is a technique used when we need to run multiple computations, some of which might never halt. Instead of running them sequentially (which would get stuck if the first one loops), we *interleave* them: run all of them for 1 step, then all for 2 steps, then 3, etc. This guarantees that if *any* computation halts, we will eventually discover it.

The core problem dovetailing solves. Suppose you want to check whether *any* string is accepted by a TM M. The naive approach is:

Run M on s_1. If it accepts, great. Otherwise, run M on s_2. Then s_3. ...

But if M loops on s_1, you are stuck forever and never try s_2. Dovetailing avoids this:

	s_1	s_2	s_3	s_4	$\cdots$
Round 1	1 step				
Round 2	2 steps	2 steps			
Round 3	3 steps	3 steps	3 steps		
Round 4	4 steps	4 steps	4 steps	4 steps	
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

In round i, we simulate M on each of $s_1, \ldots, s_i$ for i steps. If M accepts any s_j in t steps, then round $\max(j, t)$ will discover this. The key insight: **no single non-halting computation can block progress on the others.**

4.4.1 Application 1: RE is Closed Under Kleene Star

Theorem 4.3: RE is Closed Under Kleene Star

If $L \in \text{RE}$, then $L^* \in \text{RE}$.

Proof. Let M be a TM that recognizes L. We construct a TM N that recognizes L^*.

Recall that $w \in L^*$ means w can be split into pieces $w = w_1 w_2 \cdots w_k$ where each $w_j \in L$ (and $k \geq 0$). The challenge is: there are finitely many ways to split w (but potentially many), and for each split, running M on a piece might loop forever.

Dovetailing handles this by trying all splits with increasing time budgets.

TM N recognizing L^*

$N =$ "On input w:
1. If $w = \varepsilon$, **accept**. (The empty string is always in L^*.)
2. For $i = 1, 2, 3, \ldots$:
   a. For every way to split w into $1 \leq k \leq |w|$ pieces $w_1, w_2, \ldots, w_k$:
      (i) Simulate M on each w_j for at most i steps (using the universal TM U).
      (ii) If M accepted *all* of $w_1, \ldots, w_k$ within i steps, **accept**.

"

Correctness.

- If $w \in L^*$: there exists a split $w = w_1 \cdots w_k$ where each $w_j \in L$. Since M recognizes L, it accepts each w_j in some finite number of steps t_j. Let $t = \max(t_1, \ldots, t_k)$. When $i \geq t$, the algorithm tries this particular split and runs M long enough on each piece to accept. So N accepts.
- If $w \notin L^*$: no split has all pieces in L, so N never accepts and runs forever.

This is RE behavior, so $L^* \in \text{RE}$. □

Remark 4.2: Why We Need Dovetailing Here

Without dovetailing, a naive approach would be: "for each split, run M on w_1, then w_2, ..." But M might loop on w_1 for a bad split, blocking us from ever trying other splits. Dovetailing ensures we cycle through *all* splits with increasing time budgets, so a good split is eventually found if one exists.

4.4.2 Application 2: Recognizing NE_{TM}

Definition 4.5: NE_{TM}

$$\text{NE}_{\text{TM}} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) \neq \emptyset\}$$

This is the language of all TM descriptions whose language is *non-empty* — i.e., M accepts at least one string.

Theorem 4.4: $\text{NE}_{\text{TM}} \in \text{RE}$

The language NE_{TM} is recursively enumerable.

Proof. We construct a TM N that recognizes NE_{TM}. The idea: we want to check "does there exist *any* string w that M accepts?" We cannot just try strings one by one (we might get stuck on the first one), so we dovetail.

TM N recognizing $\mathrm{NE}_{\mathrm{TM}}$

$N =$ "On input $\langle M \rangle$, where M is a TM:

1. For $t = 1, 2, 3, \ldots$:
   a. Let $s_1, s_2, \ldots, s_t$ be the first t strings over M's input alphabet (in minlex order).
   b. For each s_j $(1 \leq j \leq t)$:
      (i) Simulate M on s_j for t steps using the universal TM U.
      (ii) If M accepts s_j within t steps, **accept**.

"

Correctness.

- If $L(M) \neq \emptyset$: there exists some s_j that M accepts in t^* steps. When $t \geq \max(j, t^*)$, the string s_j is in the list and the simulation runs long enough to accept it. So N accepts $\langle M \rangle$.
- If $L(M) = \emptyset$: M accepts nothing, so no simulation ever leads to acceptance. N runs forever.

This is RE behavior, so $\mathrm{NE}_{\mathrm{TM}} \in \mathrm{RE}$. □

Remark 4.3: Dovetailing Table for $\mathrm{NE}_{\mathrm{TM}}$

Visually, the dovetailing proceeds as:

	s_1	s_2	s_3	s_4	$\cdots$
$t = 1$	M on s_1 for 1 step				
$t = 2$	M on s_1 for 2 steps	M on s_2 for 2 steps			
$t = 3$	M on s_1 for 3 steps	M on s_2 for 3 steps	M on s_3 for 3 steps		
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Each cell is an independent simulation. If *any* cell results in acceptance, N accepts immediately. The table grows diagonally, ensuring every (string, time budget) pair is eventually reached.